building a financial news pipeline that ran itself for four months

aug 22, 2026

this is a writeup of summarizedfinance.net: what it does, how it is built, the failure modes i hit, and why the thing it was optimised for did not work. it processed around 11,400 articles and published roughly 4,900 of them, running unattended on a three-hour cycle. i want to describe the engineering honestly, including the parts that were wrong, because the interesting content is mostly in the diagnosis rather than the architecture.

the short version: the system works, the content strategy does not, and those are separable problems.

what it does

every three hours the pipeline pulls eight rss feeds, extracts article text where it can, discards near-duplicate stories, asks a language model to score each remaining article for sentiment and impact, generates a short summary for the highest-impact subset, and publishes the result as a static page. the site also carries an aggregate view: a histogram of sentiment across the corpus and a daily mean going back ninety days.

the data model is deliberately boring. one sqlite file, three tables, no external services beyond the model api. everything is derived from `raw_articles`, which is the only table that ever receives new rows from the outside world.

shape of the pipeline

the stages run in a fixed order inside a single process:

```
fetch → dedupe → score → select → extract → enrich → render
```

the ordering is not arbitrary. deduplication sits immediately after fetch and before anything that costs money, because five outlets covering the same central bank decision produce five near-identical rows, and paying to score all five is pure waste. extraction sits after selection rather than before it, because fetching and parsing full article html is the slowest stage and there is no point doing it for articles that will never be published. each stage is idempotent and reads its own work queue out of sqlite, so a crash halfway through a cycle costs at most one cycle.

ingestion

`feedparser` against eight sources, with a uniqueness constraint on article url to make re-fetching harmless. feeds are polled far more often than they update, so a typical cycle sees between zero and ten genuinely new items per source.

the feed list went through several revisions and the revisions taught me more than the initial choice did. i started with a mix that included wall street journal, financial times, marketwatch, yahoo finance and business insider, reasoning that better outlets would produce better summaries. that reasoning was correct and irrelevant, because i could not access the text. more on this below.

deduplication

titles are normalised by lowercasing, stripping punctuation, collapsing whitespace, and removing a stopword list that includes both ordinary english function words and news boilerplate ("says", "reports", "live", "updates", "breaking"). the normalised strings are compared with `difflib.sequencematcher`, which implements a ratcliff-obershelp style similarity ratio, against every article fetched in the previous 48 hours.

two parameters matter. the similarity threshold is 0.85, which is high enough that it only catches genuinely the same story rather than the same topic. i tested lower thresholds and they started eating legitimate variations, for instance two different companies reporting earnings on the same day with structurally identical headlines. the window is 48 hours, which approximately matches a news cycle; beyond that, a similar headline is usually a genuine follow-up story rather than a duplicate.

there is no embedding model here and i do not think one is warranted. wire services propagate near-verbatim headlines, so the duplicates i care about are lexically close by construction. semantic similarity would catch more, and would also start catching things i do not want caught.

the backfill pass over historical data was $o(n^2)$ inside the time window, which is fine at a few thousand rows and would not be at a few hundred thousand. if this ever needed to scale, the obvious fix is blocking on a coarse key before doing pairwise comparison.

scoring and selection

every surviving article gets a sentiment score in [-1, 1] and an impact score in [0, 1] from the model, working from the headline and the rss abstract. the top articles by impact are then rescored, with additional calls per item, and the mean is taken. this is a crude variance reduction: single-call scores on the same input drift by more than i was comfortable with, and the articles that actually get published are a small enough subset that paying twice for them is affordable.

i want to be careful about what this does and does not establish. the scores are internally consistent enough to be useful for ranking and for the aggregate charts. i have never validated them against anything external. whether the daily mean sentiment correlates with actual index returns is an open question i have not answered, and i would not claim the scores measure market sentiment in any rigorous sense until i have.

extraction, and the paywall problem

full article text is fetched with `trafilatura`, which does a good job of stripping navigation and boilerplate from news html. it is gated behind an explicit source allowlist.

the allowlist exists because of a failure that took me embarrassingly long to see. extraction against a paywalled or login-walled source does not fail loudly. `trafilatura` receives a perfectly valid html document, because the paywall interstitial *is* a valid html document, and returns whatever text it finds there. sometimes that is a subscription prompt. sometimes it is the first two sentences of the article followed by a subscription prompt. the pipeline records a successful extraction and moves on.

the downstream effect is that the model receives a 150 character stub instead of a 3,000 character article, and dutifully produces a summary anyway. language models do not refuse to write when handed insufficient input. they pad. that padding is fluent, plausible, and empty, and it is very hard to spot by reading individual outputs, because each one reads fine in isolation.

the diagnostic that mattered

i noticed the symptom, which was that some published articles felt thin, long before i understood the cause. my first instinct was to blame the summarisation prompt. that instinct was wrong, and the thing that showed it was wrong was a single query grouping input length against output length by source:

```
SELECT source,
       COUNT(*)                                        AS n,
       AVG(LENGTH(COALESCE(content_text, summary)))    AS avg_input,
       AVG(LENGTH(ai_summary))                         AS avg_output
FROM raw_articles
WHERE ai_summary_status = 'ok'
  AND ghost_post_id IS NOT NULL
GROUP BY source
ORDER BY avg_output ASC;
```

the result was not a gradient. it was two clearly separated populations:

source	n	avg input	avg output
investing_news	3	0	290
yahoo_finance	9	0	331
marketwatch_top	80	140	345
ft_home	94	94	357
cbs_moneywatch	4	165	357
theguardian_business	9	1,198	497
bbc_business	27	2,979	944
cnbc_topnews	187	3,672	1,030

sources where extraction succeeded were producing summaries of roughly 950 to 1,030 characters. sources where it did not were producing 290 to 360, from inputs of 0 to 165 characters. the two clusters do not overlap.

the rows with `avg_input = 0` are the ones that should worry anybody. those are articles where the pipeline had no text at all, not even an rss abstract, and the model produced a 290 character summary regardless. those summaries were generated from a headline and nothing else. they were, functionally, hallucinations with a plausible byline, and they were live on a public site.

two things follow from this. the first is operational: the prompt was never the problem, the input was, and no amount of prompt engineering would have fixed a source that returns nothing. the second is more general. when a model sits in the middle of a pipeline, output quality is a function of input quality, and if you only ever inspect outputs you will misattribute the failure. profiling the relationship between the two is cheap and i should have done it months earlier.

the quality floor

the fix was a publication threshold on summary length, set at 500 characters. that number is not a guess. it sits in the empty region between the two clusters in the table above, so it separates them cleanly without needing to encode anything about which source produced what.

applying it retroactively flagged 552 already-published articles, about 11 percent of everything on the site. i unpublished them. articles from sources that could be extracted properly were reset so the next cycle would re-fetch, re-summarise and republish them from full text. articles from sources that structurally could not be extracted were marked as permanently skipped, since re-running them would produce the same empty output.

i used a soft unpublish (setting posts to draft) rather than a hard delete for most of them, purely so the decision was reversible. publicly the effect is identical, which raised the obvious question of what deleting 552 pages does to search indexing.

the answer, which i want to state carefully because i initially got it wrong in my own head, is that it depends entirely on the http status code. a 404 on a specific url is a normal, expected signal: that page is gone, drop it from the index, carry on crawling the rest of the site. a 5xx or a connection timeout is a signal about the *site*, and sustained 5xx responses cause search engines to back off the whole domain. i had previously killed a deployment badly enough to produce the second case, and the recovery was slow. removing 552 pages cleanly produced 31 entries in the "not found" report and no measurable effect on anything else. the great majority had never been crawled at all, so 404ing them was invisible.

storage and scheduling

sqlite, one file, wal off, no orm. every stage does its own connection and its own explicit sql. for a workload where the writer is a single sequential process running every three hours, anything more than this is decoration.

scheduling is a systemd `oneshot` service plus a timer, rather than cron, because i wanted journal integration and the ability to inspect the last run's state. this produced my favourite bug in the project.

the pipeline had been running fine for weeks. i expanded the feed list from three sources to eight. new articles stopped appearing. nothing in the application logs indicated an error, because from the application's point of view there was no error; there was no log line at all past a certain point in the cycle.

the cause was `timeoutstartsec`, which defaults to 90 seconds in systemd but is raised to a larger default for `oneshot` units, and which i had never set. with three feeds a cycle finished comfortably inside it. with eight feeds, and therefore more extraction and more model calls, cycles began exceeding it, and systemd was killing the process mid-run and marking the unit failed. the unit status said `result: timeout` in plain text the entire time. i had been reading application logs and not unit status.

the fix is one line in a drop-in file. the lesson, which i actually take seriously, is that when a scheduled job silently stops producing output, the supervisor's opinion of the job is the first thing to check, not the job's own logs. a process that has been sigtermed does not get to write a log line explaining that it was sigtermed.

serving, version one

the site originally ran on ghost, with the pipeline pushing posts through the admin api using jwt auth. that choice was a reaction to a previous failure.

an earlier version of this project was a client-rendered single page app. it was a good app and it was effectively invisible to search engines. per-route metadata injected client side does not reliably reach a crawler, article pages with no server-rendered links pointing at them do not get discovered, and the various patches for this (prerendered snapshots, a noscript block of article links in the shell) each helped a little and none of them fixed the shape of the problem. moving to a cms that served real html at real urls fixed it in one step.

so ghost earned its place initially. what it did not do was keep earning it. i never used the editor, because everything was published through the api. i never used the membership features, the newsletter, or the themes beyond light modification. what i had was a node process and a mysql server sitting in the request path so that a pipeline could write rows that another process would later turn into html.

the moment this became obvious was when i lost the admin password. recovering it meant generating a bcrypt hash and writing it directly into the mysql `users` table, at which point i could not log in anyway, because ghost sends a device verification email and the box had no working mail transport. i had a cms i could not administer, publishing content i generated elsewhere, backed by a database i only ever touched to fix the cms.

serving, version two

i replaced all of it with a static generator: about 800 lines of python and jinja2 that reads sqlite and writes html files. roughly 5,000 pages, 25 mb, built in a single pass. nginx serves a directory.

a few decisions worth spelling out.

builds are atomic. output is written to a staging directory and swapped into place with a rename, with the previous version kept until the swap succeeds. a failed build therefore never leaves a partially written site on disk, which matters when the thing writing the site runs unattended at three in the morning. this is also the one place the design has a sharp edge: because the staging directory is created as a sibling of the output directory, the *parent* directory must be writable by the build user, which is not the intuitive permission to grant and which broke the first deployment attempt.

the charts are server-rendered svg, computed in python and emitted as markup. there is no charting library, no cdn request, and no javascript anywhere on the site. a histogram is a handful of `rect` elements and a sentiment trend is a `polyline`; both are a few dozen lines of coordinate arithmetic. the benefit beyond speed is that the chart is in the html, so it exists for anything that reads the page without executing scripts.

theming is one dictionary of css custom property values at the top of the generator. every colour and font in the site resolves from it, including the chart colours, which means restyling is an edit to fifteen lines rather than a hunt through templates.

the migration, and the 44 slugs

moving hosts meant moving 4,943 published urls without losing any of them, since those urls were the only thing the site had accumulated that was worth anything.

the naive approach is to generate each page's path from `raw_articles.slug`, which is the slug the pipeline computed before publishing. this is wrong, and the way it is wrong is instructive. ghost enforces slug uniqueness itself, and when two articles produced the same slug, it silently appended `-2` to the second one and published it at that url. it did not tell the pipeline, and the pipeline never recorded it. so for 44 articles, the url that was actually live and actually indexed did not exist anywhere in my database.

building from `slug` would have produced two articles writing to the same output path, one of them silently overwriting the other, and 44 urls returning 404 with no error anywhere in the build.

the fix was to export the cms `posts` table, join it back to the pipeline database on the stored post id, and write the real published slug into a new `live_slug` column. rendering then reads `coalesce(live_slug, slug)`. the verification step is a set difference between the urls the generator produced and the urls the cms reported as published, and it needs to be empty:

```
find . -name index.html -printf '/%h/\n' | sed 's|/\./|/|' | sort -u > built.txt
sort -u published-urls.txt > cms.txt
comm -13 built.txt cms.txt          # must be empty
```

it came back with exactly one entry, an `/about/` page that existed in the cms and had no counterpart in the pipeline database, which i then added to the generator. 4,943 of 4,943 article urls matched.

the general point is that a derived system can hold state the source of truth does not know about, and a migration is exactly when that costs you. reconciling against the live system, rather than regenerating from first principles, is the difference between a clean cutover and a slow leak of 404s that you discover weeks later in a coverage report.

why none of this worked

the site currently gets close to zero organic search traffic. this is worth explaining properly, because it is the most useful thing in the project.

when i moved to a fresh domain, indexing behaved beautifully. within a few days google had indexed 265 pages, average position was 8.7, which is bottom of the first page, and the site was getting around 1,900 impressions and 30 clicks a week. coverage reports showed 882 urls discovered and queued for crawling. it looked like the beginning of a growth curve.

it was not. fresh domains get a temporary allowance, and what happens next is a reassessment. over the following months indexed pages decayed. a `site:` query now returns single digits, all of them from the initial window.

the structural reason is simple and i do not think there is an engineering answer to it. the site publishes model-written summaries of other outlets' reporting. when someone searches for a story that cnbc broke, a search engine that sends them to my paraphrase instead of to cnbc has served them worse. scaled content policies target exactly this shape, and they are, in this instance, correct. every improvement i made was real (deduplication, a quality floor, structured data, clean urls, fast static pages) and none of them addressed the actual problem, because the actual problem was that the content was derivative by construction.

i spent a long time treating a content problem as a technical problem, mostly because the technical problem was the one i knew how to work on. that is the failure mode worth naming.

what the corpus is actually for

the one genuinely original artefact here is not the summaries. it is the dataset: 11,400 articles with timestamps, sources, and model-assigned sentiment and impact scores, collected continuously over months. nobody else has that particular table.

the obvious next piece of work, which i have not done yet, is to find out whether the scores mean anything. running a classical baseline such as vader or finbert over the same corpus and measuring agreement with the model scores would establish whether the llm is doing something a cheaper method could do. testing the daily mean sentiment against index returns over the same period would establish whether it carries any signal at all. both are a weekend of work, and both produce a real finding regardless of which way the result goes. a negative result here is publishable in the sense that matters: it tells you something you did not know.

that is a different project from the one i built, and it is the one that should have been the project all along.